

Lập trình .NET (ASP.NET Core Web API)

**Buổi 6: CRUD với Entity
Framework Core, LINQ và
quan hệ dữ liệu**



1. Lý thuyết

1.1. LINQ

1.2. CRUD

1.3. Quan hệ dữ liệu

1.4. Include

2. Thực hành

2.1. CRUD Student

2.2. Quan hệ Student-Class

LINQ là gì?



1.1. LINQ (tt)

LINQ là gì?

Language-Integrated Query (LINQ) cung cấp cú pháp truy vấn dữ liệu trực tiếp trong C#.

Trong EF Core, LINQ được biên dịch tự động thành các câu lệnh SQL (như “SELECT”, “WHERE”, “ORDER BY”) để chạy dưới CSDL.

Hỗ trợ Methods Syntax (Dùng nhiều nhất): “.Where()”, “.Select()”, “.OrderBy()”, “.FirstOrDefault()”,...

Create (Thêm mới)

Sử dụng `Add()` và bắt buộc phải gọi `SaveChanges()` (hoặc `SaveChangesAsync()`) để đẩy câu lệnh `INSERT` xuống CSDL.

```
var student = new Student
```

```
{
```

```
    Name = "Nguyen Van A",
```

```
    Age = 20
```

```
};
```

```
_context.Students.Add(student);
```

```
await _context.SaveChangesAsync(); // Lưu xuống SQL
```

Read (Đọc dữ liệu)

Sử dụng LINQ để truy vấn (tạo ra câu lệnh `SELECT`).

// Lấy toàn bộ

```
var allStudents = await _context.Students.ToListAsync();
```

// Lấy theo ID

```
var student = await _context.Students.FindAsync(id);
```

// Lọc dữ liệu có điều kiện (WHERE)

```
var adults = await _context.Students  
    .Where(s => s.Age >= 18)  
    .ToListAsync();
```

Update (Cập nhật)

Nguyên tắc: Truy vấn dữ liệu ra -> Thay đổi thuộc tính -> Lưu lại (EF Core có cơ chế Change Tracking để sinh câu lệnh `UPDATE`).

```
var student = await _context.Students.FindAsync(id);  
if (student != null)  
{  
    student.Age = 21; // EF Core sẽ tự nhận biết trường Age bị thay đổi  
    await _context.SaveChangesAsync();  
}
```

Delete (Xóa)

Nguyên tắc: Truy vấn dữ liệu ra -> Gọi hàm 'Remove()' -> Lưu lại.

```
var student = await _context.Students.FindAsync(id);
```

```
if (student != null)
```

```
{
```

```
    _context.Students.Remove(student);
```

```
    await _context.SaveChangesAsync();
```

```
}
```

Best Practice: Luôn dùng async/await với DB

- Các thao tác I/O như đọc/ghi Database rất chậm so với tốc độ CPU.
- Luôn sử dụng các hàm bất đồng bộ (“ToListAsync”, “FirstOrDefaultAsync”, “SaveChangesAsync”) để tránh block luồng xử lý (thread) của Web Server, giúp ứng dụng chịu tải tốt hơn.

1.3. Quan hệ dữ liệu

Mô hình phổ biến nhất: **One-to-Many** (1-N).

Ví dụ: 1 Lớp (Class) có nhiều Sinh viên (Student).

```
public class Class
{
    public int Id { get; set; }
    public string? ClassName { get; set; }
    // Navigation Property
    public ICollection<Student> Students { get; set; }
}
```

→ 1 : N

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    // Foreign Key
    public int ClassId { get; set; }
    // Navigation Property
    public Class Class { get; set; }
}
```

1.4. Include

Eager Loading với Include

Mặc định EF Core KHÔNG tải dữ liệu bảng liên quan — cần Include() để JOIN.

1.4. Include (tt)

Ví dụ:

```
// Chỉ lấy thông tin Student (Class sẽ bị null)
var students = await _context.Students.ToListAsync();
{
    "id": 1,
    "name": "Nguyen Van A",
    "age": 20,
    "classId": 5,
    "class": null // THIẾU DỮ LIỆU
}
```

```
// Lấy Student kèm theo thông tin của Class
var studentsWithClass = await _context.Students
    .Include(s => s.Class)
    .ToListAsync();
{
    "id": 1,
    "name": "Nguyen Van A",
    "age": 20,
    "classId": 5,
    "class": {
        "id": 5,
        "className": "CNTT-K6A"
    }
}
```

=> Tương đương SQL: `SELECT s.*, c.* FROM Students s INNER JOIN Classes c ON s.ClassId = c.Id`

1.4. Include (tt)

Lỗi thường gặp: Object Cycle

VẤN ĐỀ:

Khi sử dụng “Include”, đối tượng “Class” chứa danh sách “Students”, mỗi “Student” lại trỏ ngược về “Class”... gây ra vòng lặp vô tận (Object Cycle) khi serialize ra JSON.



Cách khắc phục:

Dùng **DTO** (để cắt vòng lặp) hoặc cấu hình bỏ qua vòng lặp tham chiếu trong thư viện JSON (không khuyến khích bằng dùng DTO).

1.4. Include (tt)



Ghi nhớ: Theo dõi truy vấn SQL

Trong quá trình phát triển, hãy luôn quan sát cửa sổ Console — EF Core in ra các câu lệnh SQL thật mà nó sinh ra.

▣ Kiểm soát hiệu suất

Phát hiện ngay khi EF Core sinh ra câu SQL quá phức tạp hoặc chạy N+1 query không cần thiết.

▣ Tránh lỗi N+1 Query

N+1 xảy ra khi Load dữ liệu liên quan trong vòng lặp — 1 query cha + N query con = thảm họa performance.

▣ Debug nhanh

Biết chính xác câu SQL sinh ra là gì — copy trực tiếp sang SQL Server Management Studio để test/tune.

◆ **Cách bật:** Add logging trong appsettings.json hoặc dùng LogTo() trong OnConfiguring.

1.4. Include (tt)



Ghi nhớ: Theo dõi truy vấn SQL

Trong quá trình phát triển, hãy luôn quan sát cửa sổ Console — EF Core in ra các câu lệnh SQL thật mà nó sinh ra.

▣ Kiểm soát hiệu suất

Phát hiện ngay khi EF Core sinh ra câu SQL quá phức tạp hoặc chạy N+1 query không cần thiết.

▣ Tránh lỗi N+1 Query

N+1 xảy ra khi Load dữ liệu liên quan trong vòng lặp — 1 query cha + N query con = thảm họa performance.

▣ Debug nhanh

Biết chính xác câu SQL sinh ra là gì — copy trực tiếp sang SQL Server Management Studio để test/tune.

◆ **Cách bật:** Add logging trong appsettings.json hoặc dùng `LogTo()` trong `OnConfiguring`.

2.1. CRUD Student

1. Viết Controller cho “Student” với các endpoint CRUD cơ bản tương tác với Database thực tế.
2. Thêm Entity “Class”, cấu hình quan hệ 1-N, chạy lại Migration.
3. Demo việc không dùng “Include” (bị thiếu dữ liệu) và dùng “Include” (có đủ dữ liệu).
4. Áp dụng DTO để giải quyết bài toán Object Cycle khi trả về API.

2.2. Bài thực hành

1. Hoàn thiện các API endpoints cơ bản: “GET /api/students”, “GET /api/students/{id}”, “POST /api/students” lưu dữ liệu xuống SQL Server.
2. Viết API cập nhật “PUT /api/students/{id}” sử dụng Entity Framework Core.
3. Thiết lập quan hệ 1-N: “Class” (Lớp) và “Student” (Sinh viên). Mỗi sinh viên thuộc 1 Lớp. Cập nhật Database bằng Migration.
4. Sửa hàm lấy danh sách sinh viên “GET /api/students”, sử dụng “Include” để trả về thêm thông tin Tên Lớp của sinh viên đó thông qua DTO.
5. Viết một endpoint “GET /api/classes/{id}/students” trả về thông tin của một Lớp và danh sách toàn bộ các sinh viên thuộc Lớp đó, ứng dụng DTO chuẩn xác để không bị lỗi Object Cycle.

